МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ПРАКТИЧЕСКОМУ ЗАНЯТИЮ № 1

Тема: «От текста заказчика к функциям Python: анализ требований, блок-схемы и спецификации»

Параметр	Значение
Дисциплина	Программирование и прототипирование ИТ-решений
Направление подготовки	09.03.03 Прикладная информатика (профиль «Бизнес и системная аналитика»)
Время выполнения	1 пара (2 академических часа / 90 минут)
Формат работы	В парах со сменой ролей (Системный аналитик + Разработчик)
Инструменты	VS Code, Python 3.11+, Markdown / Mermaid (или draw.io / бумага), GitHub (через браузер)

1. Цель работы

В прошлых лабораторных работах вы писали весь код в одном файле сверху вниз: и ввод данных (input), и формулы, и вывод (print). В реальных ИТ-проектах так не делают: большую задачу всегда делят на маленькие независимые функции (это называется функциональной декомпозицией).

При этом работу делят два специалиста:

1. **Системный аналитик (СА)** — общается с заказчиком, переводит его пожелания в чёткую таблицу правил и рисует блок-схему (карту алгоритма).
2. **Разработчик (Dev)** — смотрит на схему аналитика и создаёт в Python спецификации функций: продумывает их названия, входные параметры, типы данных и описание (docstring), а внутри пока ставит заглушку ... (многоточие) или pass.

Наглядный пример: «Было» vs «Стало»

КАК БЫЛО (весь код в одной куче):

```
q = float(input("Введите дебит: "))
w = float(input("Введите обводненность: "))
q_oil = q * (1 - w / 100)
print("Нефть:", q_oil)
```

КАК СТАЛО (спецификация функции, которую мы учимся делать сегодня):

```
def calc_oil_volume(q_liq: float, w_pct: float) -> float:
    """Считает объём чистой нефти по формуле: Qн = Qж * (1 - W / 100).

    Args:
        q_liq (float): Дебит жидкости (от 0 до 1000 м3/сут).
        w_pct (float): Обводнённость (от 0 до 100 %).
```

```
    Returns:
        float: Дебит чистой нефти (м3/сут).
    """
    ...  # Заглушка: саму формулу напишем позже, главное — согласовать каркас!
```

2. Краткая шпаргалка (Глоссарий)

Термин	Простыми словами
Декомпозиция	Разбиение одной большой программы на маленькие функции, где каждая функция делает ровно одно дело (одна проверяет ввод, другая считает формулу, третья определяет статус).
Блок-схема	Рисунок алгоритма из фигур: овал — Начало/Конец; параллелограмм — Ввод/Вывод; прямоугольник — Формула; ромб — Условие (if/while), из которого всегда выходят две стрелки: Да и Нет.
Сигнатура функции	Заголовок функции: def имя(параметр: тип) -> тип_ответа:.
Type Hints (типы)	Подсказки типов данных: float — дробное число, int — целое, str — строка, bool — True/False.
Docstring (докстринг)	Описание функции в тройных кавычках """...""" сразу на первой строчке внутри def.
Заглушка (... или pass)	Пустое тело функции, чтобы Python не выдавал ошибку, пока код внутри ещё не написан.

3. План работы на пару (90 минут) и чек-лист сдачи

Распределите роли в паре: Студент 1 — Системный аналитик (CA), Студент 2 — Разработчик (Dev).

Время	Этап работы	Что делает пара
00–10 мин	Старт	Сложите ваши номера по журналу N = N1 + N2 и посчитайте свои входные числа. Создайте папку prac1.
10–35 мин	Параллельная работа	Аналитик заполняет таблицу требований и строит блок-схему. Разработчик создаёт файл specs.py и подставляет порог пары.
35–60 мин	Синхронизация и расчёт	Посчитайте ответ на калькуляторе (Прогноз), запустите python specs.py и сравните с ответом программы (Факт).

60–80 мин	Два челленджа (смена ролей)	Поменяйтесь местами за ПК и выполните Челленджи № 1 и № 2 из Раздела 6.
80–90 мин	Загрузка на GitHub и сдача	Загрузите файлы README.md и specs.py на GitHub через браузер (Раздел 7) и покажите преподавателю.

Что должно быть готово к концу пары (Чек-лист):

- Заполненная таблица требований (4 строки) и блок-схема алгоритма.
- Рабочий файл specs.py с 4 функциями (в двух реализован код, в двух — заглушки со спецификациями).
- Заполненная таблица сверки «Прогноз на калькуляторе | Факт программы».
- Выполненные Челленджи № 1 и № 2 и ссылка на загруженный репозиторий GitHub.

4. Вариант А (Базовый): «Оценка рентабельности скважины»

4.1. Что просит заказчик (Технолог цеха)

> *«Ребята, нам нужно переписать ваш старый калькулятор в виде понятных функций. Оператор вводит дебит жидкости Qж (допустимо от 0 до 1000 м³/сут) и обводнённость W (от 0 до 100 %). Если ввёл число вне границ — переспрашиваем. Сначала считаем объём чистой нефти: Qн = Qж * (1 - W / 100). Затем переводим объём в массу в тоннах: Мн = Qн * 850 / 1000 (где 850 кг/м³ — плотность нефти). В конце определяем статус скважины: если Мн < M_min — выдаём 'Убыточна (стоп)', если от M_min до 3 * M_min — 'Рентабельна (база)', а если 3 * M_min и больше — 'Высокодебитная (приоритет)'.»*

4.2. Посчитайте свои индивидуальные числа (Вариант А)

Сложите ваши номера по журналу N = N1 + N2 и вычислите 3 параметра (знак % — остаток от деления):

```
N     = N1 + N2                 # сумма номеров студентов пары по журналу
Qж    = 50 + 3 * (N % 25)       # тестовый дебит жидкости, м³/сут
W     = 15 + 2 * (N % 35)       # тестовая обводнённость, %
M_min = 8.0 + 0.5 * (N % 10)    # минимальный порог рентабельности для вашей пары, т/сут
```

Шаг 1 (Делает Аналитик): Таблица требований и блок-схема

Заполните таблицу требований, выписав правила из рассказа технолога:

Элемент задачи	Описание из текста заказчика	Ограничения / Формула (заполнить)
Входные данные	Дебит жидкости Qж, обводнённость W	Qж: от 0 до 1000; W: от ... до ...
Расчёт объёма нефти	Перевод жидкости в чистую нефть Qн	Qн = ...
Расчёт массы нефти	Перевод м³ нефти в тонны Мн	Мн = ...
Правило статуса	Сравнение Мн с порогом M_min	3 статуса: ...

Вставьте в README.md код блок-схемы Mermaid (или зарисуйте её на листе / в draw.io, указав ваш порог M_min):

```
```mermaid
flowchart TD
    Start([Начало]) --> InputQ[/Ввод Qж и W/]
    InputQ --> CheckValid{Qж в 0..1000 И W в 0..100?}
    CheckValid -- Нет --> InputQ
    CheckValid -- Да --> CalcVol[Расчёт объёма нефти: Qн = Qж * 1 - W/100]
    CalcVol --> CalcMass[Расчёт массы нефти: Мн = Qн * 850 / 1000]
    CalcMass --> Cond1{Мн < M_min?}
    Cond1 -- Да --> Status1[Статус: Убыточна - стоп]
    Cond1 -- Нет --> Cond2{Мн < 3 * M_min?}
    Cond2 -- Да --> Status2[Статус: Рентабельна - база]
    Cond2 -- Нет --> Status3[Статус: Высокодебитная - приоритет]
    Status1 --> Output[/Вывод: Qн, Мн, Статус/]
    Status2 --> Output
    Status3 --> Output
    Output --> End([Конец])
```
```

Шаг 2 (Делает Разработчик): Файл спецификаций specs.py

Создайте файл specs.py, скопируйте код ниже и обязательно замените M_MIN_THRESHOLD на ваш M_min:

```
"""Спецификации функций модуля оценки рентабельности скважины (Вариант А)."""

OIL_DENSITY: float = 850.0
M_MIN_THRESHOLD: float = 10.5  # <-- ЗАМЕНИТЕ НА ВАШ M_min!


def read_valid_param(prompt: str, min_val: float, max_val: float) -> float:
    """Запрашивает у пользователя число в цикле, пока оно не попадёт в [min_val; max_val].

    Args:
```

```
        prompt (str): Текст подсказки при вводе.
        min_val (float): Минимально допустимое значение.
        max_val (float): Максимально допустимое значение.
    Returns:
        float: Проверенное число.
    """
    ...


def calc_oil_volume(q_liq: float, w_pct: float) -> float:
    """Считает суточный объём чистой нефти по формуле: Qн = Qж * (1 - W / 100).

    Args:
        q_liq (float): Дебит жидкости, м3/сут (0..1000).
        w_pct (float): Обводнённость, % (0..100).
    Returns:
        float: Объёмный дебит нефти Qн, м3/сут.
    """
    ...


def calc_oil_mass(q_oil: float, density: float = OIL_DENSITY) -> float:
    """Переводит объём нефти (м3/сут) в массу (т/сут) по формуле: Мн = Qн * density / 1000.

    Args:
        q_oil (float): Объёмный дебит нефти, м3/сут.
        density (float): Плотность нефти, кг/м3 (по умолчанию 850.0).
    Returns:
        float: Массовый дебит нефти Мн, т/сут.
    """
    ...


def evaluate_profitability(m_oil: float, m_min: float = M_MIN_THRESHOLD) -> str:
    """Определяет статус рентабельности скважины по массе нефти Мн.

    Правило:
    - Мн < m_min -> 'Убыточна (стоп)'
    - m_min <= Мн < 3 * m_min -> 'Рентабельна (база)'
    - Мн >= 3 * m_min -> 'Высокодебитная (приоритет)'

    Args:
        m_oil (float): Массовый дебит нефти, т/сут.
        m_min (float): Порог рентабельности, т/сут.
    Returns:
        str: Текстовый статус скважины.
    """
    if m_oil < m_min:
        return "Убыточна (стоп)"
    elif m_oil < 3 * m_min:
        return "Рентабельна (база)"
    else:
        return "Высокодебитная (приоритет)"


if __name__ == "__main__":
    # Посчитайте на калькуляторе вашу массу нефти Мн и подставьте сюда:
    my_test_mass = 25.0
```

```
    status = evaluate_profitability(my_test_mass)
    print(f"Масса нефти: {my_test_mass:.2f} т/сут (порог {M_MIN_THRESHOLD}) -> Статус:
{status}")
```

Шаг 3 (Вместе): Сверка «Прогноз — Факт»

Посчитайте вручную на калькуляторе Qн, Мн и ожидаемый статус для ваших Qж и W, запустите python specs.py и заполните таблицу:

Показатель	Прогноз Аналитика (вручную)	Вывод программы (specs.py)	Совпало?
Объём нефти Qн, м³/сут	(ваше число)	(появится в Челлендже 1)	Да / Нет
Масса нефти Мн, т/сут	(ваше число)	(ваше число)	Да / Нет
Статус рентабельности	(ожидаемый статус)	(вывод программы)	Да / Нет

5. Вариант Б (Продвинутый): «Аварийная диагностика скважины»

(Выполняется парами с уверенным знанием синтаксиса Python. Все функции пишем также в одном файле specs.py без вложенных папок).

5.1. Что просит заказчик (Главный инженер)

> *«Нам нужен модуль диагностики по 4 датчикам: дебит жидкости Qж (0..1000 м³/сут), обводнённость W (0..100 %), пластовое давление Рпл (5..60 МПа) и забойное давление Рзаб (1..60 МПа). Логика из 3 шагов: 1) Считаем перепад давления (депрессию) dP = Рпл - Рзаб и продуктивность J = Qж / dP (если dP <= 0, делить нельзя — возвращаем J = 0.0). 2) Определяем код состояния по очереди проверок: если dP < 0 -> 'BLOWOUT'; иначе если dP <= 0.5 -> 'IDLE'; иначе если W >= 85.0 -> 'WATERED'; иначе если J < J_norm -> 'CLOGGED'; иначе -> 'NORMAL'. 3) По коду состояния через match-case выдаём команду диспетчеру.»*

5.2. Посчитайте свои индивидуальные числа (Вариант Б)

```
N      = N1 + N2                  # сумма номеров студентов пары по журналу
Qж     = 40 + 4 * (N % 20)        # дебит жидкости, м³/сут
W      = 20 + 3 * (N % 28)        # обводнённость, %
Рпл    = 22.0 + 0.5 * (N % 15)    # пластовое давление, МПа
Рзаб   = 14.0 + 0.4 * (N % 18)    # забойное давление, МПа
J_norm = 4.5 + 0.25 * (N % 12)    # норматив продуктивности для вашей пары
```

Шаг 1 (Аналитик): Таблица требований и блок-схема

Заполните таблицу требований по образцу Варианта А и сохраните в README.md блок-схему:

````
```mermaid
flowchart TD
    Start([Вход: Qж, W, Рпл, Рзаб]) --> CalcDP[Депрессия: dP = Рпл - Рзаб]
    CalcDP --> CheckDP{dP > 0?}
    CheckDP -- Нет --> SetJZero[J = 0.0]
    CheckDP -- Да --> CalcJ[Продуктивность: J = Qж / dP]
    SetJZero --> Diag1{dP < 0?}
    CalcJ --> Diag1
    Diag1 -- Да --> CodeBlow[Код: BLOWOUT]
    Diag1 -- Нет --> Diag2{dP <= 0.5?}
    Diag2 -- Да --> CodeIdle[Код: IDLE]
    Diag2 -- Нет --> Diag3{W >= 85%?}
    Diag3 -- Да --> CodeWat[Код: WATERED]
    Diag3 -- Нет --> Diag4{J < J_norm?}
    Diag4 -- Да --> CodeClog[Код: CLOGGED]
    Diag4 -- Нет --> CodeNorm[Код: NORMAL]
    CodeBlow --> End([Вывод кода и директивы диспетчеру])
    CodeIdle --> End
    CodeWat --> End
    CodeClog --> End
    CodeNorm --> End
```
````

Шаг 2 (Разработчик): Файл спецификаций specs.py (Вариант Б)

```
"""Спецификации модуля аварийной диагностики скважины (Вариант Б)."""

W_CRITICAL: float = 85.0
DP_MIN_ACTIVE: float = 0.5
J_NORM_DEFAULT: float = 5.5  # <-- ЗАМЕНИТЕ НА ВАШ J_norm!


def validate_telemetry(q_liq: float, w_pct: float, p_res: float, p_bhp: float) -> bool:
    """Проверяет, что все 4 датчика находятся в допустимых физических границах.

    Args:
        q_liq (float): Дебит жидкости (0..1000 м3/сут).
        w_pct (float): Обводнённость (0..100 %).
        p_res (float): Пластовое давление (5..60 МПа).
        p_bhp (float): Забойное давление (1..60 МПа).
    Returns:
        bool: True, если все показатели в норме, иначе False.
    """
    ...


def calc_hydraulics(q_liq: float, p_res: float, p_bhp: float) -> tuple[float, float]:
    """Считает депрессию dP = p_res - p_bhp и продуктивность J = q_liq / dP.
    Если dP <= 0, то J принимается равным 0.0 (защита от деления на ноль).

    Args:
        q_liq (float): Дебит жидкости, м3/сут.
```

```
        p_res (float): Пластовое давление, МПа.
        p_bhp (float): Забойное давление, МПа.
    Returns:
        tuple[float, float]: Пара чисел (dP, J).
    """
    ...


def determine_status_code(
    drawdown: float, w_pct: float, prod_index: float, j_norm: float = J_NORM_DEFAULT
) -> str:
    """Определяет код состояния скважины по приоритетной цепочке условий.

    Args:
        drawdown (float): Депрессия dP, МПа.
        w_pct (float): Обводнённость W, %.
        prod_index (float): Коэффициент продуктивности J.
        j_norm (float): Нормативный коэффициент J_norm.
    Returns:
        str: Код состояния ('BLOWOUT', 'IDLE', 'WATERED', 'CLOGGED' или 'NORMAL').
    """
    if drawdown < 0.0:
        return "BLOWOUT"
    elif drawdown <= DP_MIN_ACTIVE:
        return "IDLE"
    elif w_pct >= W_CRITICAL:
        return "WATERED"
    elif prod_index < j_norm:
        return "CLOGGED"
    else:
        return "NORMAL"


def route_dispatcher_action(status_code: str) -> str:
    """Переводит код состояния в текстовую команду диспетчеру через match-case.

    Args:
        status_code (str): Код состояния скважины.
    Returns:
        str: Текст команды для диспетчера.
    """
    ...


if __name__ == "__main__":
    # Подставьте сюда ваши расчётные dP, W и J:
    my_dp, my_w, my_j = 8.0, 35.0, 6.5
    code = determine_status_code(my_dp, my_w, my_j)
    print(f"Вход: dP={my_dp:.2f}, W={my_w}%, J={my_j:.2f} -> Код: {code}")
```

6. Два быстрых челленджа (со сменой ролей!)

Поменяйтесь местами за компьютером: бывший Аналитик садится писать код, а бывший Разработчик проверяет его работу.

Челлендж 1 (5 минут): «Оживляем расчётную функцию и проверяем help()»

3. Добавьте в самый конец файла specs.py вызов встроенной справки: **help(calc_oil_volume)** (для Варианта А) или **help(calc_hydraulics)** (для Варианта Б). Запустите скрипт и убедитесь, что Python превратил ваш docstring в документацию (для выхода из справки в терминале нажмите клавишу Q).
4. **Оживите заглушку:** замените многоточие ... на реальный расчёт return ... (в Варианте А — в функциях calc_oil_volume и calc_oil_mass; в Варианте Б — в функции calc_hydraulics). Вызовите их внизу файла для своих исходных чисел и убедитесь, что ответ совпал с вашим ручным расчётом.

Челлендж 2 (10 минут): «Внезапная правка от заказчика»

Заказчик позвонил и попросил добавить одно экстренное условие:

- **Для Варианта А:** если в evaluate_profitability(m_oil, m_min) передали m_oil == 0, нужно сразу возвращать статус 'Скважина остановлена (0 т/сут)'.
- **Для Варианта Б:** если в determine_status_code(...) передали w_pct >= 98.0, нужно раньше проверки на 85% возвращать код 'CRITICAL_WATER'.

Допишите эту ветку if в код функции, добавьте строчку про новое правило в docstring и проверьте его работу через print(...) внизу файла.

7. Как загрузить работу на GitHub через браузер и сдать отчёт

Вам не нужно настраивать Git в терминале — загрузите файлы README.md и specs.py прямо через сайт GitHub за 4 простых шага:

Шаг 7.1. Создайте репозиторий (папку проекта на GitHub)

5. Откройте сайт github.com и войдите в свой профиль.
6. В правом верхнем углу нажмите на значок «+» и выберите New repository (Новый репозиторий).
7. В поле Repository name введите название латиницей без пробелов, например: prac1-specs.
8. Убедитесь, что выбрана точка Private.
9. Прокрутите страницу вниз и нажмите зелёную кнопку Create repository.

Шаг 7.2. Загрузите файлы с компьютера

10. На странице нового репозитория нажмите на синюю ссылку uploading an existing file (в блоке Quick setup). (Если в репозитории уже есть файлы, нажмите кнопку Add file -> Upload files в правом верхнем углу).
11. Перетащите мышкой файлы вашей работы (README.md и specs.py) из папки на компьютере прямо в центральное окно браузера «Drag files here to add them to your repository» (или нажмите choose your files).
12. Дождитесь, пока оба файла появятся в списке под окном загрузки.

Шаг 7.3. Сохраните изменения (сделайте Commit)

13. Опуститесь ниже к блоку Commit changes.
14. В первом текстовом поле (где написано Add files via upload) напишите краткий комментарий: Выполнена практическая работа №1 (Аналитик + Разработчик).
15. Нажмите зелёную кнопку Commit changes внизу.

Шаг 7.4. Проверьте результат и скопируйте ссылку

16. Через пару секунд откроется главная страница вашего репозитория: убедитесь, что файлы README.md и specs.py лежат в списке, а ниже автоматически отобразилась ваша таблица и блок-схема Mermaid!
17. Скопируйте ссылку из адресной строки браузера (вида https://github.com/ваш-логин/prac1-specs) и вставьте её в отчёт.

Что включить в итоговый отчёт (один общий отчёт на пару):

18. Шапка: ФИО обоих студентов, распределение ролей (CA и Dev), расчёт N = N1 + N2, индивидуальные числа и ссылка на репозиторий GitHub.
19. Заполненная таблица требований и блок-схема алгоритма.
20. Итоговый код файла specs.py (после двух челленджей) и скриншот вывода в терминале.
21. Таблица сверки «Прогноз | Факт».
22. Краткие ответы на 4 контрольных вопроса.

Контрольные вопросы для защиты:

- Вопрос 1 (Аналитику): Чем отличается исходный рассказ заказчика от таблицы функциональных требований?
- Вопрос 2 (Аналитику): Какими фигурами на блок-схеме обозначаются: а) ввод/вывод; б) формула; в) условие if? Почему из ромба всегда должно выходить две стрелки?
- Вопрос 3 (Разработчику): Что такое docstring и что выведет help(имя_функции), если написать комментарий через решётку # над функцией вместо тройных кавычек """ внутри неё?
- Вопрос 4 (Разработчику): Зачем мы возвращаем результат из функции через return, а не просто печатаем его через print() внутри самой функции?

Частые ошибки (Проверьте себя!)

Симптом / Ошибка	Как быстро исправить
IndentationError или SyntaxError после def	Внутри функции после двоеточия обязательно должен быть отступ (4 пробела) и хотя бы docstring или многоточие ...
help(func) выводит None	Описание написано через # над функцией, а не в тройных кавычках """ первой строкой внутри функции.
Функция выводит None при print(calc_oil_volume(...))	Забыли заменить заглушку ... на команду возврата результата return формула в Челлендже № 1.
На GitHub не рисуется схема Mermaid в README.md	Проверьте, что код схемы обёрнут сверху строкой

	```mermaid (с тремя обратными кавычками), а снизу закрыт строкой ```.